Thinking 6.1 示例代码中，父进程操作管道的写端，子进程操作管道的读端。如果现在想让父进程作为“读者”，代码应当如何修改？ ■

```c
#include <stdlib.h>
#include <unistd.h>
int fildes[2];
/* buf size is 100 */
char buf[100];
int status;
int main() {
    status = pipe(fildes);
    if (status == -1) {
        printf("error\n");
    }
    switch (fork()) {
        case -1: /* Handle error */
            break;
        case 0: /* Child - writes to pipe */
            close(fildes[0]); /* Read end is unused */
            write(fildes[1], "Hello world\n", 12); /* Write data on pipe */
            close(fildes[1]); /* Child will see EOF */
            exit(EXIT_SUCCESS);
        default: /* Parent - reads from pipe */
            close(fildes[1]); /* Write end is unused */
            read(fildes[0], buf, 100); /* Get data from pipe */
            printf("parent-process read:%s\n", buf); /* Print the data */
            close(fildes[0]); /* Finished with pipe */
            exit(EXIT_SUCCESS);
    }
}
```

Thinking 6.2 上面这种不同步修改 `pp_ref` 而导致的进程竞争问题在 `user/lib/fd.c` 中的 `dup` 函数中也存在。请结合代码模仿上述情景，分析一下我们的 `dup` 函数中为什么会出现预想之外的情况？ ■

在 dup 函数中，先对 fd 执行 syscall_mem_map，然后对 data 执行 syscall_mem_map。子进程先开始执行，在 dup 和 read 之间时发生时钟中断，此时 pageref(p[1]) == 1，pageref(pipe) == 1。父进程开始执行，在 dup 函数中执行到 fd 的 map 之后，data 的 map 之前，此时 pageref(p[0]) == 1，pageref(pipe) == 1。时钟中断，子进程再次开始执行，此时执行 read 函数，判断发现 pageref(p[0]) == pageref(pipe)，误认为是写者全部结束，于是错误地返回了。

Thinking 6.3 阅读上述材料并思考：为什么系统调用一定是原子操作呢？如果你觉得不是所有的系统调用都是原子操作，请给出反例。希望能结合相关代码进行分析说明。 ■

系统调用是原子操作，因为在系统调用期间会通过汇编代码关闭时钟中断，这使得其他进程无法运行，从而保证了发起系统调用的进程能够独占整个操作系统直到系统调用完成。

Thinking 6.4 仔细阅读上面这段话，并思考下列问题

- 按照上述说法控制 `pipe_close` 中 `fd` 和 `pipe unmap` 的顺序，是否可以解决上述场景的进程竞争问题？给出你的分析过程。
- 我们只分析了 `close` 时的情形，在 `fd.c` 中有一个 `dup` 函数，用于复制文件描述符。试想，如果要复制的文件描述符指向一个管道，那么是否会出现与 `close` 类似的问题？请模仿上述材料写写你的理解。

可以解决这个问题。最初 pageref(p[0]) = 2，pageref(p[1]) = 2，pageref(pipe) = 4。仍然考虑子进程先运行，在对 p[1] 执行 close 函数时，首先解除对写端 fd 的映射，之后发生时钟中断，此时 pageref(p[0]) = 2，pageref(p[1]) = 1，pageref(pipe) = 4。父进程执行完 close(p[0]) 后，pageref(p[0]) = 1，pageref(p[1]) = 1，pageref(pipe) = 3，此时父进程开始运行时仍不满足写端关闭的条件，因此竞争问题没有出现。

在程序执行时，总满足 pageref(p[0]) ≤ pageref(pipe)。如果在 dup 中先对 fd 进行映射，此时 fd 的引用计数先增加，可能会填补两者的空隙，造成读端已满的假象（写端同理），因此也会出现与 close 类似的问题，需要调整映射的顺序。

Thinking 6.5 思考以下三个问题。

- 认真回看 Lab5 文件系统相关代码，弄清打开文件的过程。
- 回顾 Lab1 与 Lab3，思考如何读取并加载 ELF 文件。
- 在 Lab1 中我们介绍了 `data` `text` `bss` 段及它们的含义，`data` 段存放初始化过的全局变量，`bss` 段存放未初始化的全局变量。关于 `memsize` 和 `filesize`，我们在 Note 1.3.4中也解释了它们的含义与特点。关于 Note 1.3.4，注意其中关于"`bss` 段并不在文件中占数据"表述的含义。回顾 Lab3 并思考:`elf_load_seg()` 和 `load_icode_mapper()` 函数是如何确保加载 ELF 文件时，`bss` 段数据被正确加载进虚拟内存空间。`bss` 段在 ELF 中并不占空间，但 ELF 加载进内存后，`bss` 段的数据占据了空间，并且初始值都是 0。请回顾 `elf_load_seg()` 和 `load_icode_mapper()` 的实现，思考这一点是如何实现的？

下面给出一些对于上述问题的提示，以便大家更好地把握加载内核进程和加载用户进程的区别与联系，类比完成 `spawn` 函数。

关于第一个问题，在 Lab3 中我们创建进程，并且通过 `ENV_CREATE(...)` 在内核态加载了初始进程，而我们的 `spawn` 函数则是通过和文件系统交互，取得文件描述块，进而找到 ELF 在"硬盘"中的位置，进而读取。

关于第二个问题，各位已经在 Lab3 中填写了 `load_icode` 函数，实现了 ELF 可执行文件中读取数据并加载到内存空间，其中通过调用 `elf_load_seg` 函数来加载各个程序段。在 Lab3 中我们要填写 `load_icode_mapper` 回调函数，在内核态下加载 ELF 数据到内存空间；相应地，在 Lab6 中 `spawn` 函数也需要在用户态下使用系统调用为 ELF 数据分配空间。 ■

对于 BSS 段在 ELF 文件中不占用空间但在内存中占用空间的问题，是在 elf_load_seg 函数中实现的。在该函数中，对于 bin_size < sgsize 的情况，直接用 0 填充，使得程序占据的空间达到 sgsize，这样就确保了 BSS 段中的所有值被初始化为 0。

Thinking 6.6 通过阅读代码空白段的注释我们知道，将标准输入或输出定向到文件，需要我们将其 `dup` 到 0 或 1 号文件描述符（fd）。那么问题来了：在哪步，0 和 1 被"安排"为标准输入和标准输出？请分析代码执行流程，给出答案。 ■

在user/init.c中加入下列代码

```
if((r = opencons()) ! = 0){
    user_panic("opencons: %d", r);
}
// stout
if((r = dup(0, 1)) , 0){
    user_panic("dup: %d", r);
}
```

Thinking 6.7 在 shell 中执行的命令分为内置命令和外部命令。在执行内置命令时 shell 不需要 fork 一个子 shell，如 Linux 系统中的 cd 命令。在执行外部命令时 shell 需要 fork 一个子 shell，然后子 shell 去执行这条命令。

据此判断，在 MOS 中我们用到的 shell 命令是内置命令还是外部命令？请思考为什么 Linux 的 cd 命令是内部命令而不是外部命令？ ■

在 MOS 中，我们使用的 shell 命令是外部命令，因为在解析和执行命令时会 fork 一个子 shell。而 Linux 的 cd 命令之所以是内部命令，是因为 cd 命令不是一个独立的可执行文件，而是由 shell 的一个内置函数实现的。当解析到 cd 命令时，只需要调用该函数即可，这样可以提高系统的运行效率。

可以观察到两次 spawn，分别对应 3805 和 4006 进程，也就是 ls.b 命令和 cat.b 命令的两个进程。

Thinking 6.8 在你的 shell 中输入命令 `ls.b | cat.b > motd`。

- 请问你可以在你的 shell 中观察到几次 spawn？分别对应哪个进程？
- 请问你可以在你的 shell 中观察到几次进程销毁？分别对应哪个进程？

■

可以观察到四次进程销毁，分别对应 3805、4006、3004、2803 四个进程，即 ls.b 命令进程、cat.b 命令进程、管道 fork 出的子 shell 进程以及最初 main 函数中 fork 出的子 shell 进程。